

臺北市立建國高級中學 114 學年度科學展覽

作品說明書

科 別：電腦與資訊學科

組 別：高級中等學校組

作品名稱：基於 WGAN-GP 的字體風格遷移研究

關 鍵 詞：WGAN-GP、中文字體生成、zi2zi

編 號：

基於 WGAN-GP 的字體風格遷移研究

摘要

為解決中文字體生成門檻過高的問題，本研究提出一套基於 WGAN-GP 的少樣本字體風格遷移架構，解決傳統 GAN 容易發生的模式崩潰等問題。研究核心發現有三，首先我們證實了同步變換增強能有效地利用漢字部件重複的特性，提升模型對陌生字符的泛化能力。其次，針對瓶頸層尺寸對生成效果之影響，本研究透過 t-SNE 與 Linear Probe 實驗進行系統性的分析，得出 4×4 與 8×8 為最佳設定。最後則發現 WGAN-GP 使用的 Wasserstein 距離能對結構提供穩定的梯度指引，使我們能夠移除易造成模糊的 L1 像素損失生成結構精準且更為銳利的圖像，相較過往對 L1 損失的依賴是有價值的發現！最終本研究將上述經驗總結套用至中文字體，取得理想效果驗證了此架構的潛力，為降低字體生成門檻提供了具價值的技術路徑。

壹、前言

一、研究動機

近年來，隨著技術的進步以及設計個人化需求的增加，市面上及社群軟體討論中出現了多種英文字體的生成工具（如：Calligraphr、Fontself 等）。透過這些工具，使用者只需手寫 52 個字母大小寫及數字，便能自動生成完整的個人化字體。

然而在中文漢字領域卻較少見類似的應用，由於漢字筆畫結構複雜、字數龐大等特性，若要求使用者手寫數千個常用字（參照「常用國字標準字體表」約為 5000 字）才能生成字體，實用性將大幅降低。因此，如何透過少量的手寫樣本生成風格一致的完整中文字體，成為了一個值得探索的技術挑戰。於是本研究決定開發一個實用的中文字體生成工具，讓一般使用者僅需書寫少量字符（目標 700 字以下）就能生成屬於自己的字體。

考量到中文字體生成的難度，研究初期將以英文與數字作為實驗對象，深入研究 WGAN-GP 架構及其在字體生成中的應用，並嘗試不同的資料處理及增強方式。最終，將從英文字集得到的實驗結果歸納，進一步應用於未來實作中文字體的生成任務，以降低個人化中文字體的製作門檻。

二、 研究目的

（一）建構基於 WGAN-GP 的字體生成模型

探討並實作 WGAN-GP (Wasserstein GAN with Gradient Penalty) 架構，藉此解決傳統 GAN 在訓練過程中容易產生的模式崩潰 (Mode Collapse) 與梯度消失 (Gradient Vanishing) 問題，建立一個穩定且高品質的字體圖像生成模型。

（二）探討資料增強技術對泛化能力以及生成效果之影響

分析不同的資料增強策略 (如幾何變換、隨機遮罩) 對模型學習成效的影響。本研究將探討在訓練資料有限的情況下，如何透過資料增強處理，使模型學習字體的筆畫結構而非單純記憶像素，進而提升對陌生字符的生成品質。

（三）分析生成器瓶頸層 (Bottleneck) 尺寸對生成效果之影響

研究生成器網路中「瓶頸層 (Bottleneck)」的特徵圖空間解析度 (以下稱為「尺寸」) 對輸出結果的影響。探討在資訊壓縮精煉過程中，不同程度的壓縮如何影響生成字體的結構完整性、筆畫細節與風格還原度，並尋找最佳的尺寸設定。

（四）最佳化損失函數權重配置

探討像素損失 (L1 Loss) 與對抗損失 (Adversarial Loss, adv Loss) 之間的權重平衡。分析不同的權重如何影響生成字體的結構完整性、筆畫細節、清晰度，解決字體生成中常見的模糊或結構扭曲問題。

（五）評估跨語言字體生成之可行性

以英文與數字資料集作為基礎，驗證最佳化後的模型架構能否遷移至結構更為複雜的漢字字體生成任務。並評估在「少樣本 (Few-shot)」前提下，模型是否能依據有限的字符樣本，推論生成風格一致的其他漢字。

貳、 文獻探討

一、 中文字體生成的挑戰與 GAN 技術的應用

中文字體生成面臨著與英文字體截然不同的技術挑戰。首先是字符數量的龐大，常用漢字 (參「常用國字標準字體表」) 約為 5000 字，遠超英文及數字的 62 個字符，使得收集完整的手寫樣本成為極大的挑戰。其次是筆畫結構複雜，對模型的特徵學習能力提出了更高要求。傳統的像素級損失函數 (如 L1、L2 距離) 雖能約束結構相似性，但難以量化字體的「風格真實感」與「筆觸自然度」。而生成對抗網路 (GAN) 透過判別器評估生成品質，為這個問題提供了有效的解決方案。

二、 生成對抗網路 (Generative Adversarial Network, GAN)

Ian Goodfellow 等人 (2014) [1] 提出此非監督式學習方法，其架構由一個生成器與一個判別器組成。生成器接受噪聲或條件輸入，目標是產生近似訓練集中真實樣本的輸出。判別器則同時接收真實樣本與生成器輸出，並嘗試區分兩者。透過生成器與判別器之間的對抗訓練，兩個網路不斷更新權重，最終使生成器得以生成近似真實樣本的輸出。而後，Radford 等人 (2015) [2] 提出的 DCGAN (Deep Convolutional GAN) 將卷積神經網路 (CNN) 結構引入，發現其能有效地提取更高層次的圖像特徵，大幅提升了訓練的穩定性與生成風格圖像的品質。

三、 WGAN (Wasserstein GAN)

Arjovsky 等人 (2017) [3] 提出 WGAN，指出在原始的 GAN 中，判別器使用 JS 散度 (Jensen-Shannon Divergence) 作為評分，當生成結果分佈與真實樣本分佈不重疊時無法給出有意義的評分，容易發生模式崩潰 (Mode Collapse) 等問題。WGAN 改使用 Earth-Mover (EM, Wasserstein-1) 距離來衡量真實分佈與生成分佈的差異。當分佈不重疊仍能提供有意義的梯度，引導生成器持續學習。

四、 WGAN-GP (Wasserstein GAN with Gradient Penalty)

Gulrajani 等人 (2017) [4] 提出此改良架構。為滿足 Wasserstein 距離所需的 1-Lipschitz 連續性限制，WGAN-GP 摒棄了原本 WGAN 直接裁切權重 (Weight Clipping) 的做法，改為在損失函數中加入梯度懲罰項，約束判別器在真實樣本與生成樣本之間插值點的梯度範數接近 1，顯著提升了訓練的穩定性與收斂速度。故本研究選擇其作為基礎架構。

五、 Pix2pix (Image-to-Image Translation with Conditional Adversarial Networks)

Isola 等人 (2017) [5] 提出 Pix2pix。不同於傳統 GAN 透過隨機噪聲生成圖像，該模型是依據輸入的圖片進行生成，可用於圖像和圖像之間的映射。該研究使用了 U-Net 結構，即將 n 層模型中的第 i 層之特徵圖直接傳遞至第 $n-i$ 層，確保輸入圖片的不同維度資訊不會流失。其另一特色則是結合對抗損失與 L1 損失 (L1 Loss) 以維持風格與結構的穩定，此兩項損失之間的平衡權重，亦是本研究要討論的一個參數。

六、 zi2zi (Master Chinese Calligraphy with Conditional Adversarial Networks)

Tian (2017) [6] 發布的 zi2zi 開源專案，將 pix2pix 應用在中文字體生成。其特點是引入了類別嵌入 (Category Embedding) 機制，使字形的配對不侷限在兩字體對應，而是將大量字體作為預訓練資料使模型能學到多種字體的風格映射。不過本研究則回歸到將字體生成視為一個一對一的風格轉換任務，並沒有依賴預訓練資料對模型進行訓練。

七、 Handwritten Chinese Font Generation with Collaborative Stroke Refinement

Wen 等人 (2021) [7] 提出了此一具有協同筆畫細化 (Collaborative Stroke Refinement) 的生成架構，該研究引入了粗略分支 (Coarse Branch) 解決過細的筆劃容易在訓練過程丟失的問題。為了複用中文字中的重複結構，更提出了動態縮放增強策略 (Online Zoom-Augmentation Strategy)，透過設定的縮放與平移操作，使模型能覆蓋各種部首出現在各位置的情況，將微調資料降低至 750 個字符。Chen 等人 (2022) [8] 改良的 DG Font++ 也使用了類似的處理方法，本研究的同步變換增強即是參考自此二篇研究。

八、 小結

本研究考慮到 GAN 在圖像生成的優勢以及不穩定性，決定使用 WGAN-GP 這個較為穩定的架構進行實作。並參考前人的研究，借鑒了 Pix2pix 的編解碼器以及雙損失函數 (L1 損失 & 對抗損失) 架構，並且為了簡化模型複雜度並聚焦於 WGAN-GP 本身的潛力探討，選擇不採用 U-Net 的跳躍連接機制。另外也在 zi2zi 中看見了中文字體生成的可行性，並選擇將字體生成聚焦回一對一的風格轉換，不透過預訓練資料進行訓練。最後從 Wen 等人 (2021) [7] 的研究中得到了同步變換增強的參考，並嘗試加入了旋轉及遮罩等操作。

參、 研究設備及器材

一、 硬體

(一) 模型訓練環境 (Google Colab)

- 作業系統：Linux 6.6.105+ x86_64 (glibc 2.35)
- GPU：NVIDIA L4 GPU

(二) 本地環境 (資料處理與視覺化)

- 作業系統：Windows 10 (筆電)
- GPU：NVIDIA GeForce RTX 3050 Ti Laptop GPU

二、 軟體

(一) 模型訓練環境 (Google Colab)

- Python 3.12.12
- Pytorch 2.9.0+cu126
- PIL 11.0.0

(二) 本地環境 (資料處理與視覺化)

- Python 3.10.0

肆、 方法及過程

一、 研究流程

1. 資料處理與資料集生成製作。
2. 以數字做最初步嘗試，建立可用的 WGAN-GP 架構。
3. 引入英文字母，並透過 WGAN-GP 文獻經驗調整超參數，尋找最佳配置。
4. 針對不同的研究目標（資料處理/瓶頸層/損失權重），控制其他變因進行實驗。
5. 針對相同的目標，使用不同風格的字體進行實驗。
6. 總結上述實驗並應用至中文字體生成，評估其成果。
7. 分析各實驗結果並做結論，進一步規劃未來研究方向。

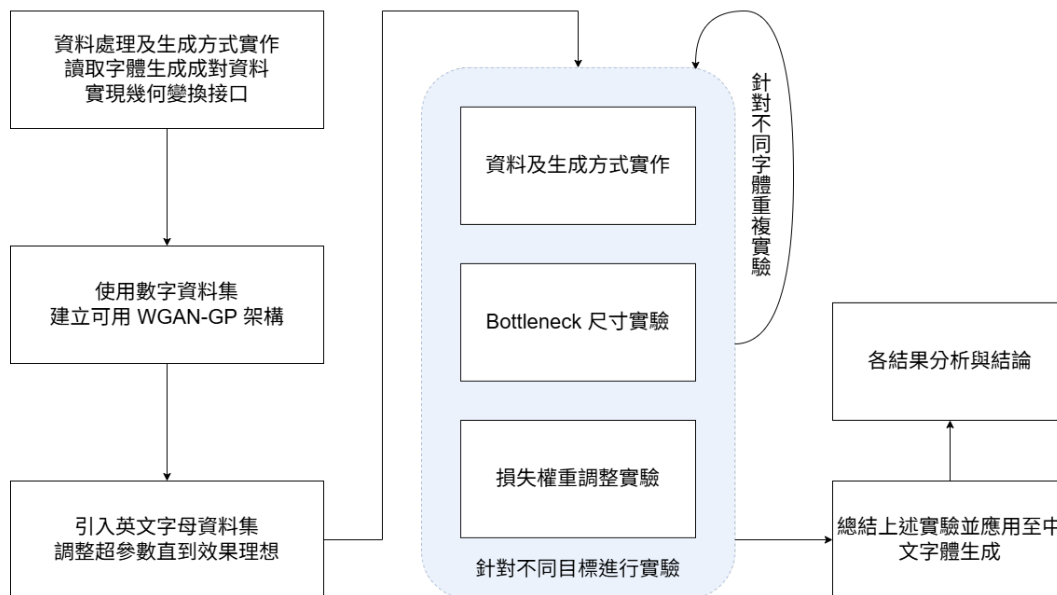


圖 1：研究流程

二、 生成字體評估標準

由於字體生成任務的結果評估須考量整體字符風格以及結構辨識度等等面向，本研究除了使用常見的量化評分外，也依據其他幾個方向進行視覺評估：

（一）量化評分

1. 結構相似性指標 (SSIM, Structural Similarity Index)，此指標綜合考量了圖像的亮度、對比度與結構。其值越高代表兩圖像越相似，是評估相似度時常用指標。
2. LPIPS (Learned Perceptual Image Patch Similarity)，為一種基於深度學習的感知指標，旨在模擬人類視覺系統對圖像相似度的判斷，較能代表兩圖片在高頻風格細節的相似性。

（二）結構完整性

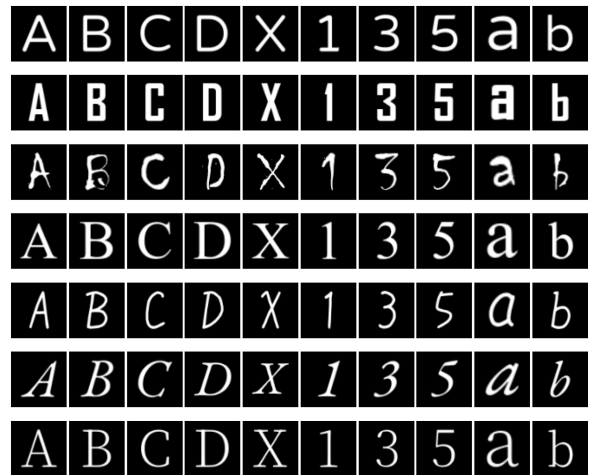
1. 字符的基本骨架是否正確（例如「A」的三角形結構、「8」的雙圓結構等）
2. 是否出現筆畫缺失或錯位等現象
3. 是否發生字符混淆（Character Confusion），即生成字符非目標字符，例如將「U」生成為「0」、「3」生成為「2」

（三）風格一致性

1. 是否保留了目標字體的特徵筆觸（襯線，傾斜等）
2. 轉折處處理（圓滑或銳利），橫豎筆畫粗細變化等是否自然並符合目標風格
3. 生成字體是否在整體視覺上接近目標字體風格

（四）清晰度

1. 筆畫邊緣是否清晰，是否出現範圍模糊（暈開）等現象
2. 是否有雜訊或不必要的像素點



三、 資料來源與處理

（一）字體來源

本研究選用如右圖（圖 2）之字體，其挑選原則為盡量包含多種風格，且變化程度從低到高皆有包含為主。

圖 2：選用字體示意圖

字體自上而下依序為 YUPearl、Agency FB、Chiller、Times、tegaki zatsu、EB Garamond、NotoSerif TC

（二）資料集產生

讀取來源字體和目標字體後，透過 pillow 套件繪製於畫布上，將其調整至 64×64 的尺寸，並調整為背景黑色、文字為白色的灰度圖。接著經過同步變換增強後生成一一對應的資料集。

（三）主動分割測試集

為了驗證模型的泛化能力並避免過度擬合（Overfitting）本研究主動將字符集劃分為「訓練集」與「測試集」。訓練時主動移除特定字符（如移除資料集中的 A、E、I、G、Z 等），模擬中文資料集缺乏完整資料的情況，並用以評估模型是否真正學會了「風格遷移」而非單純記憶圖像。

（四）動態生成機制

資料集使用動態生成 (On-the-fly Generation) 而非預先將訓練資料儲存為靜態圖片。此機制能大幅節省硬碟空間，允許在每一個 Epoch 對同一字符施加不同的隨機增強效果，相比預先生成所有資料並在每一輪重複讀取能大幅提升資料變化程度。

四、 模型架構

本研究建立了一個基於 WGAN-GP 的生成對抗網路架構，旨在解決傳統 GAN 在訓練時常見的梯度消失以及模式崩潰 (Mode Collapse) 等問題，整體架構分為生成器 (Generator) 與判別器 (Discriminator) 兩組件：

（一）生成器 (Generator)

採用「編碼器-解碼器 (Encoder-Decoder) 結構」，目的是學習來源字體圖像 (I_s) 到目標字體圖像 (I_t) 的非線性映射。

1. 編碼器 (Encoder)

由多層卷積層 (Convolutional Layers) 組成。輸入 $64 \times 64 \times 1$ 的灰階圖，逐層下採樣 (Downsampling)，壓縮特徵圖的空間解析度 (Spatial Resolution) 並增加特徵通道數 (Channels)。使用 Leaky ReLU 作為激活函數 (負斜率 $\alpha=0.2$)，相較於 ReLU 能避免死神經元 (Dead Neuron) 問題，確保有效提取字體的筆畫結構與幾何特徵。

2. 可變的瓶頸層 (Adjustable Bottleneck)

瓶頸層位於編碼器與解碼器之間，是經過編碼器壓縮後的潛在空間。本研究將此層的尺寸 ($n \times n$) 設為可調整的變因，將在後續進行討論。此外為使生成器有更大的操作空間，將同樣尺寸的高斯噪聲張量 z 與瓶頸層特徵進行拼接 (Concatenation)。

註：瓶頸層實際形狀為「寬×高×通道數」，由於下採樣邏輯相同，決定瓶頸層的空間解析度 (寬×高) 即可知道其通道數。故本篇研究若無特別註明時「尺寸」指的即是「寬×高」的尺寸。

3. 解碼器 (Decoder)

由多層轉置卷積層 (Transposed Convolution) 組成。將瓶頸層的特徵圖逐步上採樣 (Upsampling) 回原始圖像大小。層與層之間採用 ReLU 激活函數以取得更明確的特徵也避免噪點出現，最後經由 Sigmoid 函數映射至 $[0,1]$ 區間最終輸出 $64 \times 64 \times 1$ 的灰階生成圖像。

（二）判別器（Discriminator）

使用標準的 WGAN-GP 判別器，由 5 層卷積層將圖片下採樣至 1×1 ，最後一層設計為不使用激活函數直接輸出純量分數（Scalar Score），代表輸入樣本與真實分佈的 Wasserstein 距離估計值。並在損失函數中加入梯度懲罰項（Gradient Penalty），約束真實樣本與生成樣本插值點的梯度範數維持在 1 附近，確保 1-Lipschitz 連續性。[4]

（三）完整架構（如圖 3）：

1. 來源字體圖像 I_{raw-s} 與目標字體圖像 I_{raw-t} 經過同步變換增強（詳見後續章節）得到成對的 I_s 與 I_t
2. I_s 輸入至生成器編碼層，壓縮至 $n \times n$ 的瓶頸層（詳見後續章節）並將特徵與高斯噪聲 z 進行拼接
3. 解碼器將特徵上採樣，生成目標字體 $\hat{I}_t = G(I_s, z)$
4. 判別器接收 $\hat{I}_t$ 與 I_t ，計算對抗損失 L_{adv}
5. 計算 $\hat{I}_t$ 與 I_t 的 L1 距離 L_{L1}
6. 總損失由 L_{adv} 與 L_{L1} 兩者加權組合（詳見後續章節），指導生成器更新權重

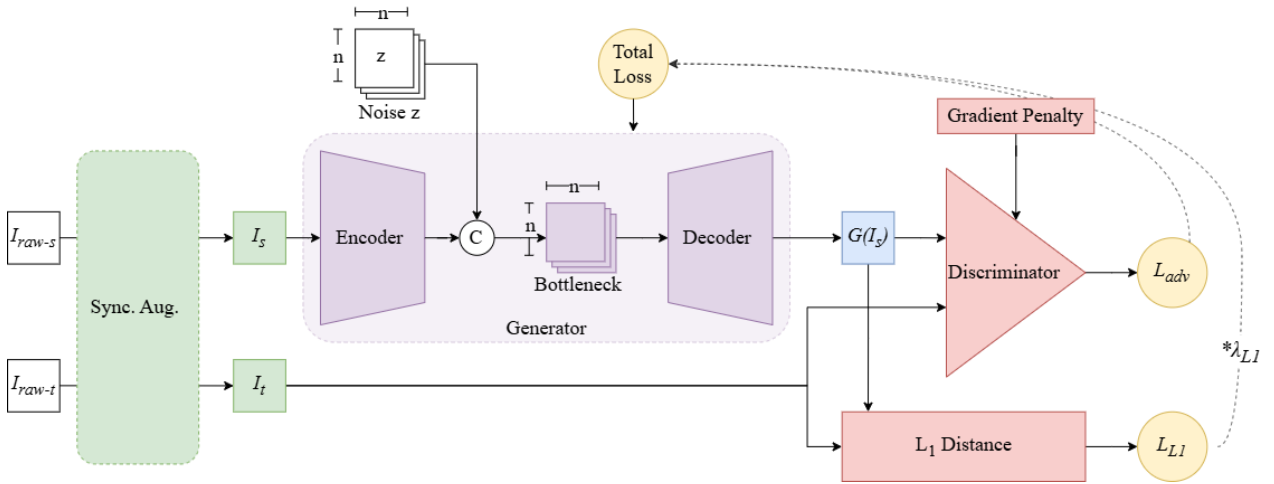


圖 3：模型架構

五、 同步變換增強

（一）幾何變換

考量手寫字體在真實書寫時具有高度的隨機性，本研究參考 Wen 等人（2021）[7] 的增強操作方法並新增了旋轉變換以及遮罩增強。由於使用一對一的圖像進行訓練，增強操作的關鍵在於對來源圖像及目標圖像施加完全相同變換，維持兩者結構的對應關係。此操作由以下參數配置：

1. 隨機縮放 (Scaling)：

模擬字體大小的變化，其值為一個數對 (S_{min}, S_{max}) 代表縮放的最小和最大比例。生成每組資料時將在此範圍內隨機應用一縮放倍數。

2. 隨機平移 (Translation)：

模擬文字在方格內的位置偏移，其值為一個數對 (T_x, T_y)，代表圖片在 x 和 y 軸的最大偏移比例（例如，若設為 (0.2, 0.2)，則圖像將在畫布寬高的 $\pm 20\%$ 範圍內隨機平移。）。

3. 隨機旋轉 (Rotation)：

模擬書寫時的傾斜角度，其值為一個正數 (θ) 代表旋轉範圍 ($\pm \theta$ 度)。生成每組資料時將在此範圍內隨機選擇一旋轉角度套用。

透過幾何增強，預期能擴增可用的數據量，讓模型遇到類似的結構能夠套用經驗，也避免過擬合現象發生。（如圖 4）

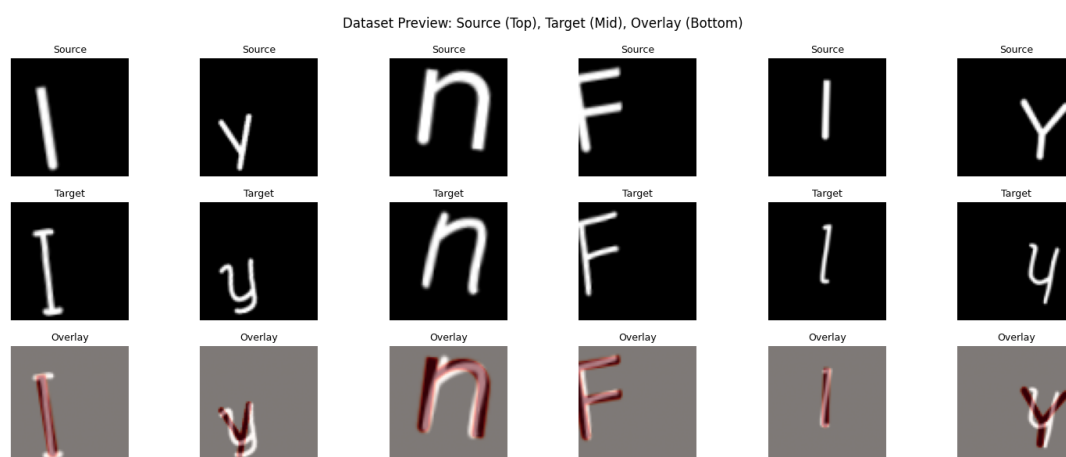


圖 4：同步變換增強示意圖

（二）隨機遮罩

為了繼續增強可用的資料量，本研究從去噪自動編碼器 (Denoising Autoencoder) 得到靈感，加入隨機遮罩機制。在訓練過程中，以固定機率 P_{mask} 做設定，即生成每組資料時，有 P_{mask} 的機率於來源及目標圖片上遮蔽同一塊隨機的矩形。此舉同樣是為了讓少量的資料能有更多樣性的變化。（如圖 5）

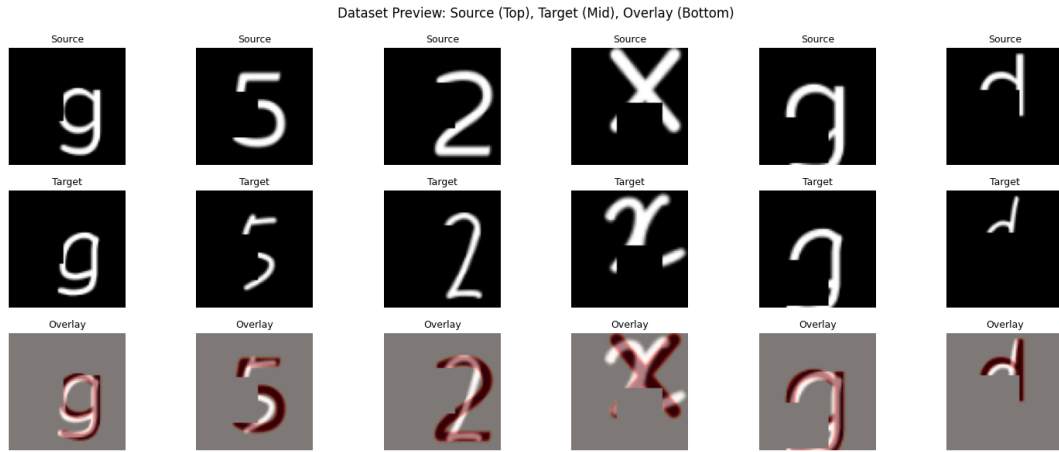


圖 5：隨機遮罩示意圖

六、可變的瓶頸層

在最經典的生成模型架構如 DCGAN [2] 中，研究者傾向將輸入資訊壓縮為一個不具空間維度的潛在向量（Latent Vector，即空間解析度為 1×1 ），例如 Pathak 等人（2016）[9] 在 Context encoders 這篇研究中將輸入壓平成長度 4000 的向量。

不過後續研究也使用了非 1×1 的瓶頸層，舉例來說 Lee 等人（2024）[10] 在 RAC-Font 這篇研究中認為將 64×64 的輸入壓縮至 16×16 是最佳設定。Zeng 等人（2021）[11] 在 StrokeGAN 這篇研究中將 $128 \times 128 \times 3$ 的輸入壓縮至 $32 \times 32 \times 256$ 。DG Font++ [8] 則將 128×128 的圖片壓縮至 16×16 。這些研究受其架構差異，瓶頸層壓縮比例各有不同，故本研究也選擇將瓶頸層尺寸設為實驗變因，探討在 $n \times n$ （ $n \in \{1, 2, 4, 8, 16\}$ ）的尺寸設定下對生成圖像品質的影響。

（一）模型架構一致性（控制變因）

在模型的控制變因上，本研究遵守「架構一致性策略（Architectural Consistency）」，亦即固定模型的卷積層數，並依照相同的下採樣邏輯（當空間解析度隨下採樣減半時，特徵通道數（Channels）相應翻倍，直到瓶頸層尺寸達到目標尺寸後繼續採樣但不改變尺寸及通道數（如圖 6）。在瓶頸層實驗中模型層數固定為 6 層，其他實驗則固定為 5 層。

Layer (type:depth-idx)	Output Shape	Param #
DynamicGenerator	[1, 1, 64, 64]	--
└Sequential: 1-1	[1, 512, 4, 4]	--
└└Conv2d: 2-1	[1, 64, 32, 32]	1,024
└└BatchNorm2d: 2-2	[1, 64, 32, 32]	128
└└LeakyReLU: 2-3	[1, 64, 32, 32]	--
└└Conv2d: 2-4	[1, 128, 16, 16]	131,072
└└BatchNorm2d: 2-5	[1, 128, 16, 16]	256
└└LeakyReLU: 2-6	[1, 128, 16, 16]	--
└└Conv2d: 2-7	[1, 256, 8, 8]	524,288
└└BatchNorm2d: 2-8	[1, 256, 8, 8]	512
└└LeakyReLU: 2-9	[1, 256, 8, 8]	--
└└Conv2d: 2-10	[1, 512, 4, 4]	2,097,152
└└BatchNorm2d: 2-11	[1, 512, 4, 4]	1,024
└└LeakyReLU: 2-12	[1, 512, 4, 4]	--
└└Conv2d: 2-13	[1, 512, 4, 4]	2,359,296
└└BatchNorm2d: 2-14	[1, 512, 4, 4]	1,024
└└LeakyReLU: 2-15	[1, 512, 4, 4]	--
└└Conv2d: 2-16	[1, 512, 4, 4]	2,359,296

圖 6：生成器編碼器架構，以 n=4 為例

（二）參數量差異

本研究雖已盡量保持架構一致，但模型在參數量上仍有不同，表 1 為各瓶頸層尺寸的模型生成器參數量（Parameters）。

表 1：瓶頸層尺寸對照生成器參數量（共 6 層捲積層）

Bottleneck size	Generator Trainable params
1×1	17,432,272
2×2	20,889,120
4×4	24,291,008
8×8	28,389,888
16×16	13,741,440

七、 損失函數設計

為了兼顧生成圖像的風格真實感與文字結構的正確性，本研究參考前人研究 [5]，令生成器 G 的總損失函數 L_{Total} 由對抗損失與像素重建損失（L1 損失）兩部分加權組成。

（一）對抗損失 (Adversarial Loss)

即為判別器輸出，代表樣本真實程度的純量分數。對於生成器而言，其目標是最小化生成圖像被判別器（Discriminator, D）評分為「假」的程度（即最大化判別器分數）。其損失函數定義為：

$$L_{adv} = -\mathbb{E}_{z, I_s} [D(G(z, I_s))]$$

其中， z 為隨機噪聲， I_s 為來源字體圖像， $G(z, I_s)$ 為生成的偽造圖像。

註：為了確保 WGAN 所需的 1-Lipschitz 條件，判別器 D 的訓練過程額外包含梯度懲罰項（Gradient Penalty），其完整損失為：

$$L_D = \mathbb{E}[D(G(z, I_s))] - \mathbb{E}[D(I_t)] + \lambda_{gp} \mathbb{E}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$$

（二）L1 像素重建損失 (L1 Reconstruction Loss)

為了確保生成的文字在筆畫結構與空間位置上與目標字體保持一致，且避免模型僅追求抽象的風格相似，故引入 L1 距離作為內容約束。此項計算生成圖像與真實目標圖像 I_t 之間像素值的絕對誤差平均值：

$$L_{L1} = \mathbb{E}_{I_s, I_t} [\|G(z, I_s) - I_t\|_1]$$

依據前人研究，此損失能提供明確的梯度指引，加速模型收斂並防止字形嚴重扭曲錯誤。[5]

（三）總目標函數 (Total Objective Function)

綜合上述兩項，生成器的總損失函數定義如下：

$$L_{\text{Total}} = L_{\text{adv}} + \lambda_{L1} \cdot L_{L1}$$

其中， λ_{L1} 為權重超參數，用於調節「高頻風格細節（由 L_{adv} 主導）」與「低頻結構保留（由 L_{L1} 主導）」之間的平衡。

伍、實驗結果與討論

一、訓練及生成結果

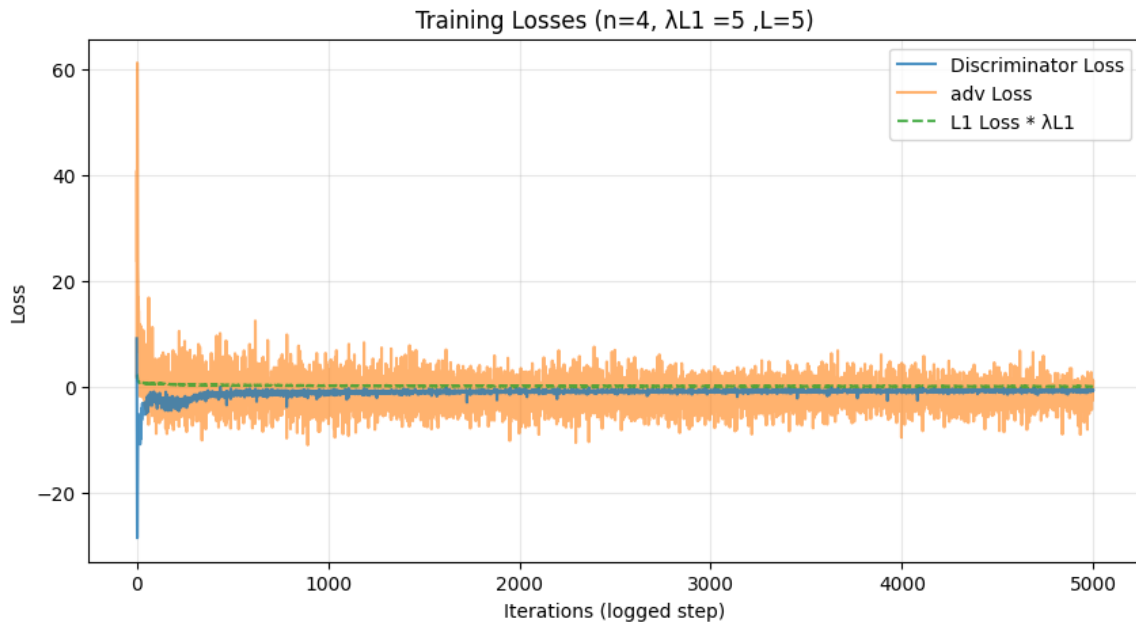


圖 7：在 WGAN-GP 架構下成功訓練之損失圖形

如圖 7 可見，L1 損失（L1 Loss）在訓練初期迅速下降，並趨近於 0。其收斂速度顯著快於對抗損失（adv Loss），且其值經過 λ_{L1} 加權後相較對抗損失仍非常小，這暗示其對訓練結果的影響可能有限，我們將於後方章節討論。

另外可見判別器損失（Discriminator Loss）迅速下降至 -30 左右後回升並穩定維持在 -1 左右的負值區間，且並未發散。這符合 WGAN 的理論趨勢，代表判別器成功在滿足 Lipschitz 限制的前提下，找到了衡量真實與生成分佈差異的標準，並持續提供有效梯度。而生成對抗損失（adv Loss）收斂後仍持續震盪，並以緩慢的速度降低震盪幅度。

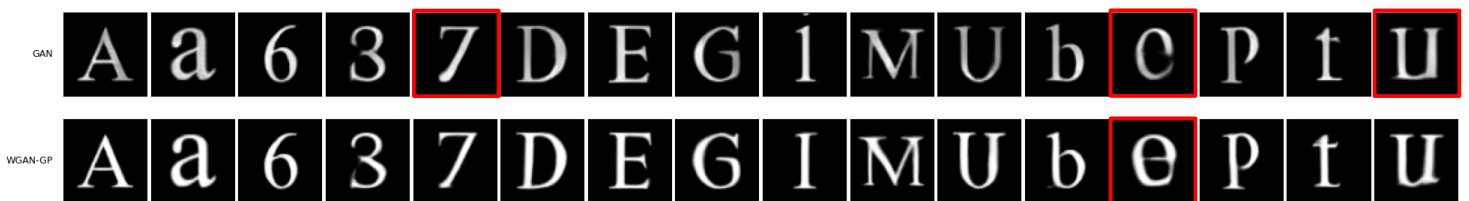


圖 8：GAN（上）與 WGAN-GP（下）架構模型生成效果對比

相較之下若使用傳統的 GAN 則較容易出現模式崩潰的問題，且生成結果會出現如上圖 8 偏灰或模糊的情況，其筆畫細節也較 WGAN-GP 差。總結來說我們成功的搭建了優於傳統 GAN 的穩定框架，其在各字體的生成效果如下圖 9 展示。

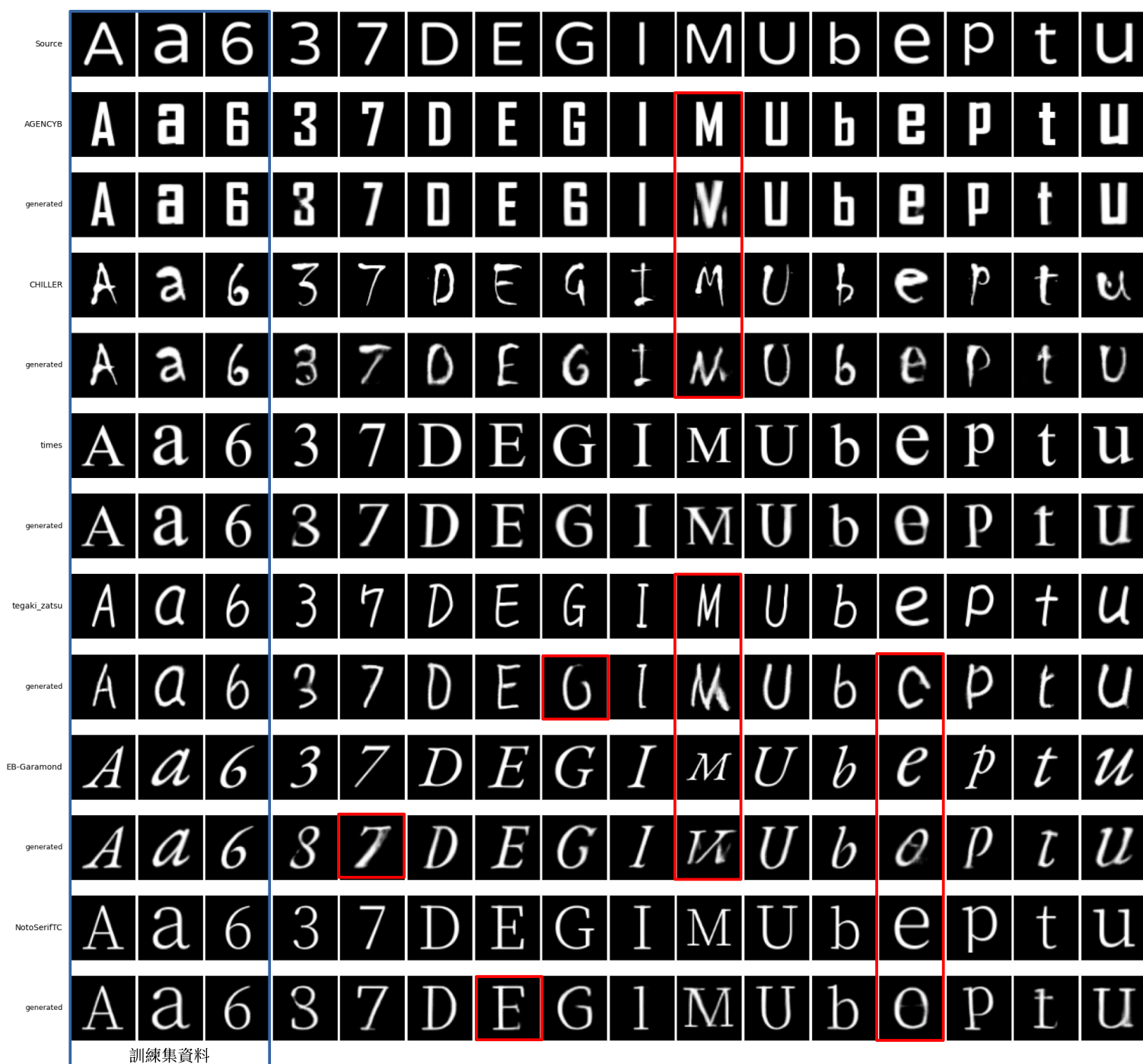


圖 9：WGAN-GP 架構模型在各字體的生成效果

模型在訓練集（圖 9 中靠左的 3 個字符為例）中在三個評估方向（結構完整、風格一致、清晰度）皆能完美模仿目標字體。測試集生成結果則受不同字符、字體的影響效果略有差異，但僅有少部分字符（M、e 等）會出現結構上的錯誤。風格一致性方面各字體皆表現優秀。清晰度的部分則沒有發現可見的噪點，不過在部分字符出現了模糊現象。

二、 同步變換增強對泛化能力以及生成效果之影響

如圖 10，縮放變換在一定的範圍有正面影響，但若縮放倍率過大會導致生成結構錯誤，而縮放倍率太小則會造成較細的筆劃被忽略。



圖 10：不同縮放變換參數 $S_{min} \sim S_{max}$ 對生成結果之影響

如圖 11，平移變換的結果與縮放類似，在合理範圍皆能顯著提高生成結果，是影響模型泛化能力的主要因素。但若移動後導致字符過多的部分被裁切，形成如圖 12 的無用數據則也會帶來負面影響。

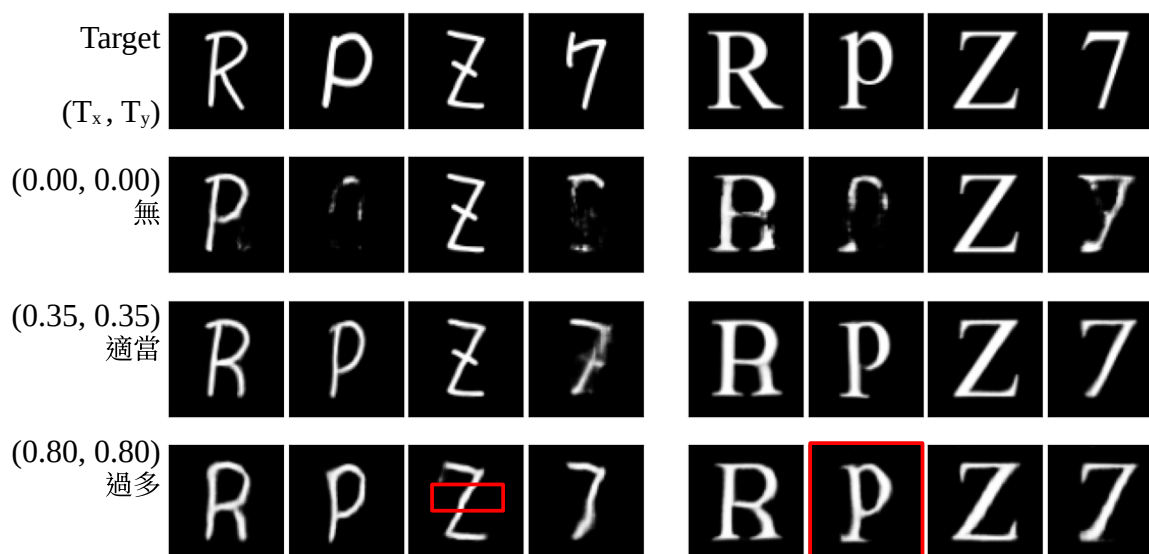


圖 11：不同平移變換參數 (T_x, T_y) 對生成結果之影響

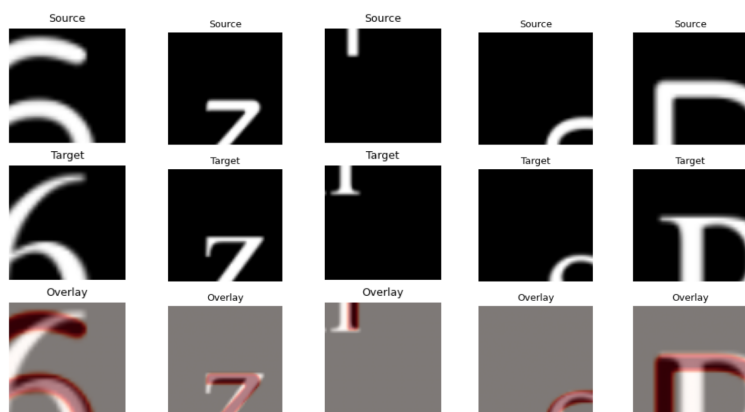


圖 12：無用數據示意圖

如圖 13，旋轉變換的效果是不顯著的，在多數情況甚至會帶來負面影響，其原因是兩種字型之間偏移的方向是固定的（例如目標字型筆畫收尾會往上偏）但在旋轉後會使得訓練資料中有些筆畫末端轉為偏左或偏右，這種矛盾的現象影響了模型生成的效果。另外有部分字體有橫向筆畫較細，直向筆畫較粗的特性，此時若加入旋轉也會導致模型混淆橫豎筆畫的粗細關係。

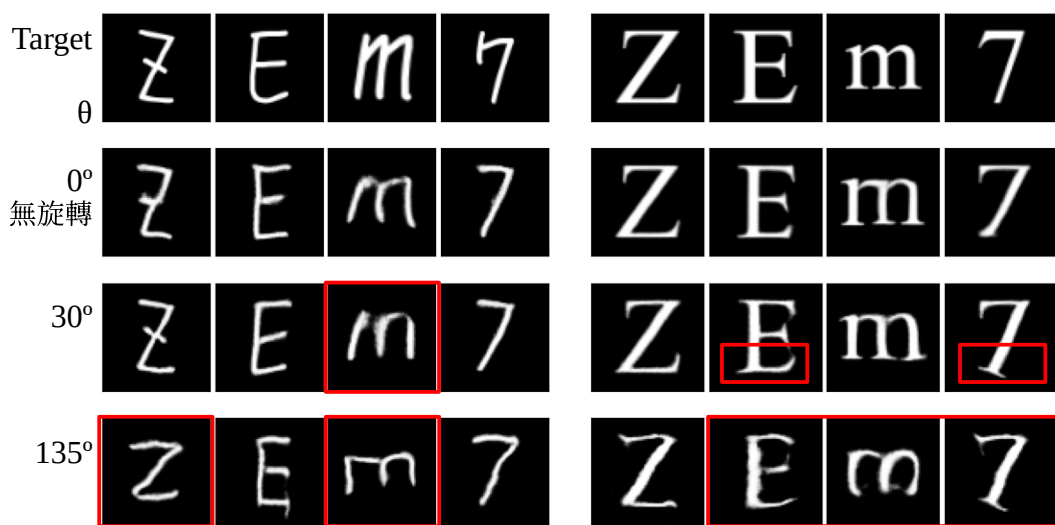


圖 13：不同旋轉變換參數 θ 對生成結果之影響

如圖 14，對於隨機遮罩增強，其在所有測試字體中皆導致生成品質下降。原因應是模型誤將遮罩截斷的地方視為筆畫末端進而影響模型對於筆畫收尾方式的理解。另一原因則是兩字體的偏移可能導致遮罩出現如圖 15 一般的錯誤遮蔽，使生成出筆畫缺失的字體。

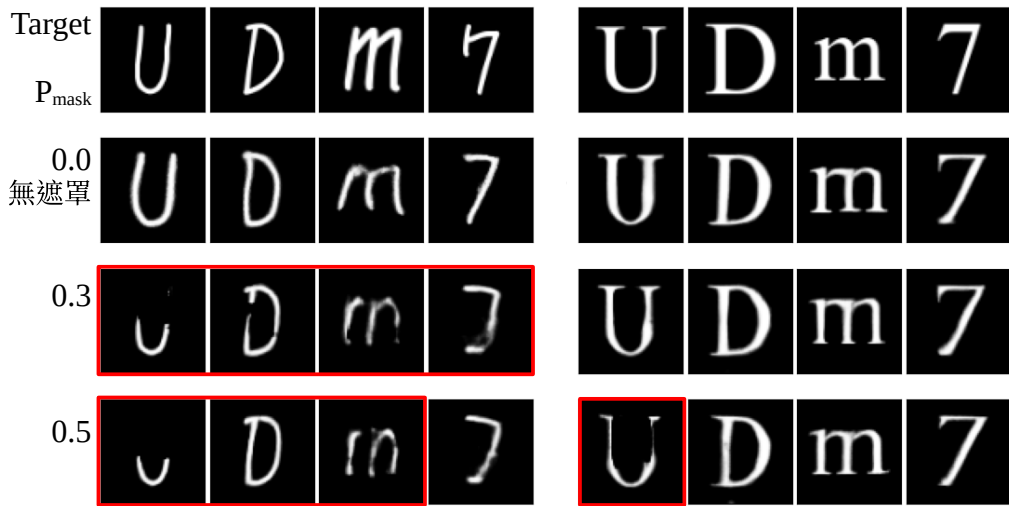


圖 14：不同遮罩機率 P_{mask} 對生成結果之影響

旋轉變換與遮罩增強在 Wen 等人 (2021) [7] 的研究並未使用，可能同樣是該研究經過實驗後的結果。而 DG Font++ [8] 的架構雖使用旋轉變換但僅將其應用至單純用來學習風格的分支，故沒有提到此變換對生成圖片結構上的負面影響。



圖 15：錯誤遮罩示意圖

三、 瓶頸層尺寸對生成效果之影響



圖 16：不同瓶頸層尺寸 $n=1,2,4,8,16$ (依序) 對字體 A 的生成效果影響 (第一列為目標)

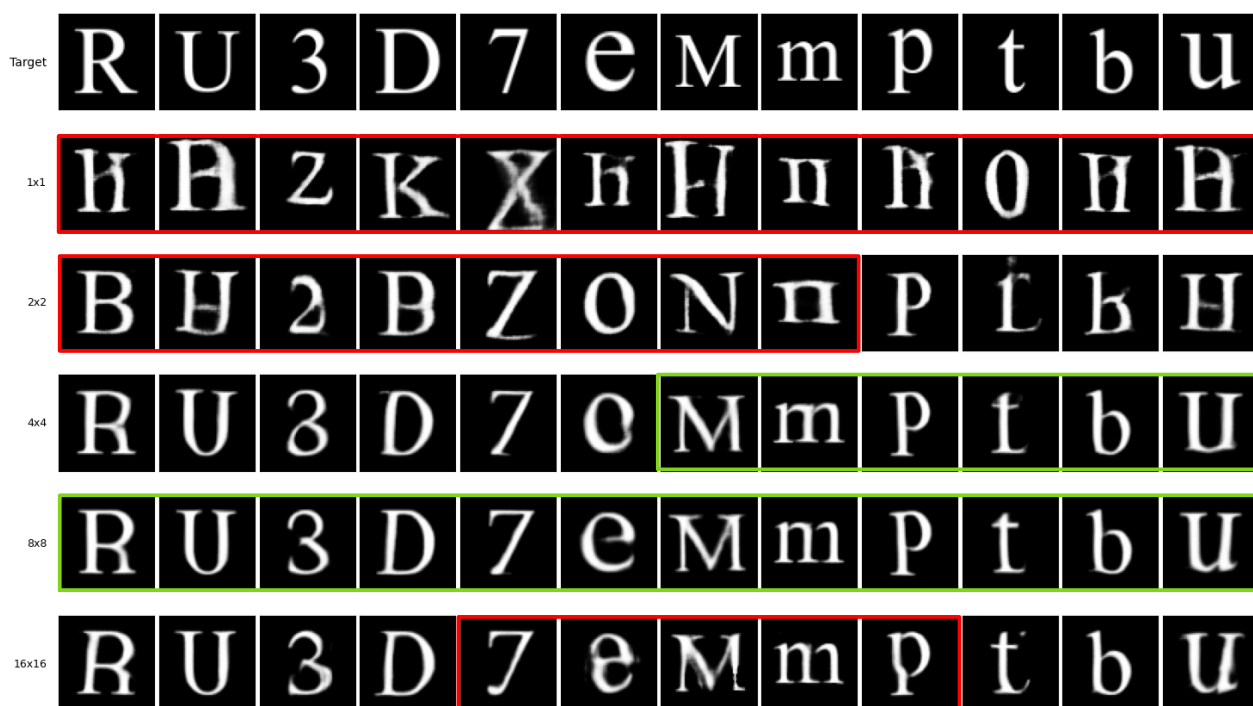


圖 17：不同瓶頸層尺寸 $n=1,2,4,8,16$ （依序）對字體 B 的生成效果影響（第一列為目標）



圖 18：不同瓶頸層尺寸 $n=1,2,4,8,16$ （依序）對字體 C 的生成效果影響（第一列為目標）

如圖 16 至 18，本研究測試了 1×1 至 16×16 等多種解析度。發現到 1×1 的瓶頸層會導致模型失去對結構的理解，出現了字符混淆的現象。綜合視覺評估以及表 2 的量化評分，顯示 4×4 及 8×8 的瓶頸層尺寸具有最好的效果。

表 2：不同瓶頸層尺寸下字體 B 及 C 於測試集之 SSIM 與 LPIPS 評分（樣本數：20000）

Bottleneck size (n×n)	SSIM-Font B↑	LPIPS-Font B↓	SSIM-Font C↑	LPIPS-Font C↓
1×1	0.615030	0.2548050	0.597888	0.243994
2×2	0.748608	0.1194810	0.714556	0.131387
4×4	<u>0.801164</u>	0.0790784	0.762089	<u>0.102000</u>
8×8	0.802506	<u>0.0769025</u>	<u>0.759602</u>	0.100094
16×16	0.792779	0.0749268	0.752425	0.102886

為了客觀驗證此發現，本研究透過一系列實驗，深入分析瓶頸層的空間保留能力，並解釋 1×1 的瓶頸層出現結構錯誤的根本原因。

（一）瓶頸層特徵視覺化

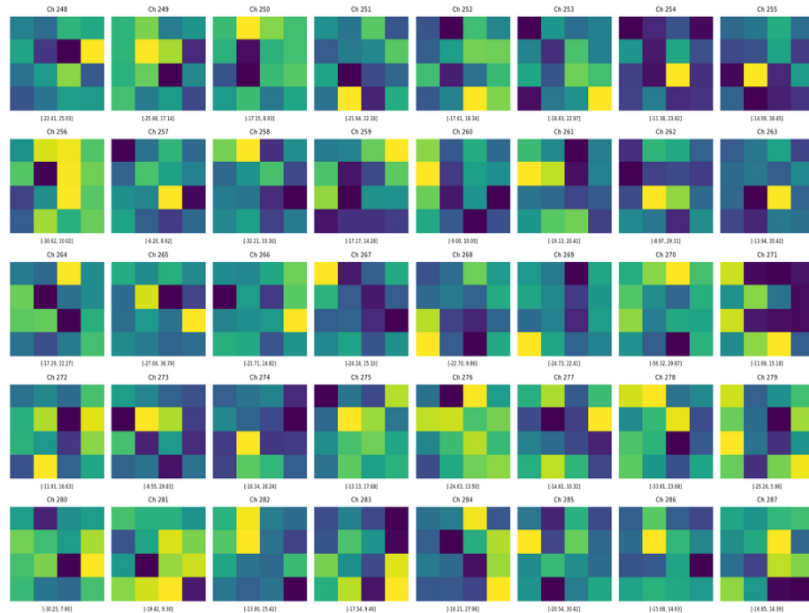


圖 19：將 4×4 的瓶頸層特徵視覺化

如圖 19，首先我們直接將瓶頸層輸出的特徵圖進行視覺化。可見瓶頸層的特徵圖呈現類似雜訊的分佈，無法直接辨識出與字符結構的關聯。這說明了深層卷積網路已將輸入圖像編碼為高度抽象的特徵（Abstract Features）。

（二）瓶頸層遮蔽測試

我們接著進行人為的遮蔽測試，即在訓練完畢後遮蔽瓶頸層的部分區塊並生成字體圖片，藉此確認 $n>1$ 時瓶頸層是否攜帶了筆畫的位置資訊。

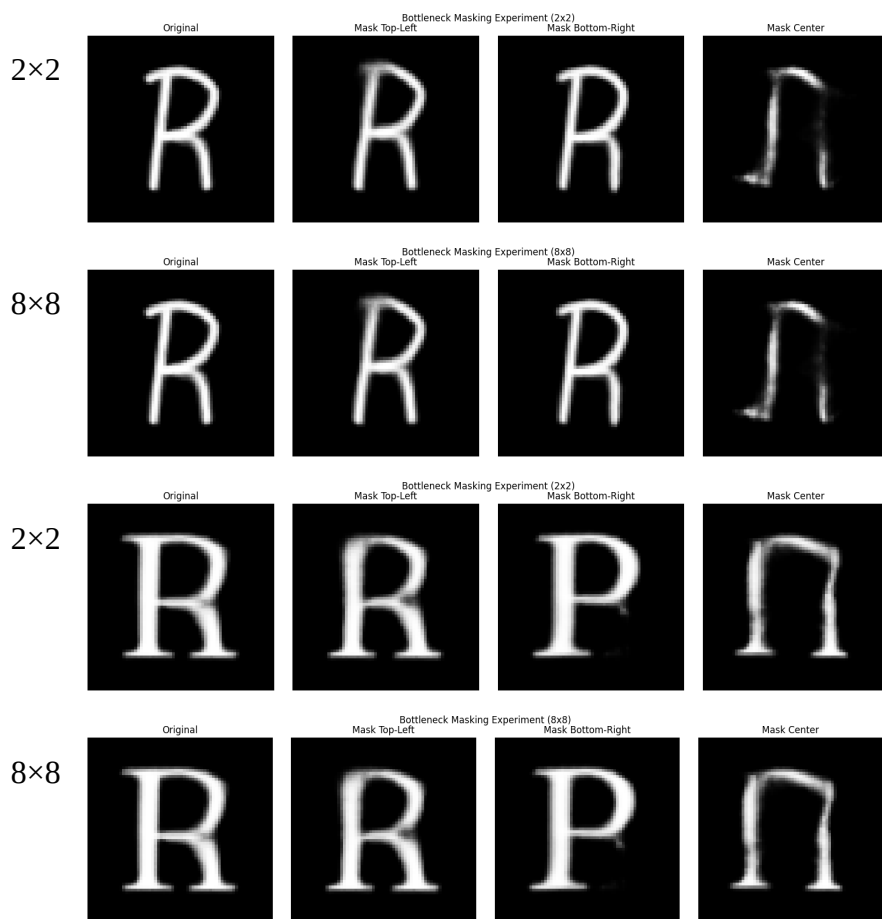


圖 20：在字體 A 及 B 中分別對 2×2 及 8×8 的瓶頸層做遮蔽測試之結果

如圖 20 顯示（左至右：無遮蔽、遮蔽左上、遮蔽右下、遮蔽中間），可得知當 $n > 1$ 時，瓶頸層確實攜帶了位置資訊，不過其攜帶的位置資訊之解析度在 $n=2$ 及 $n=8$ 的情況並沒有明顯差異（生成結果無明顯差別）。

（三）特徵降維分群測試

參考 Goldfeld 等人（2020）在 *The Information Bottleneck Problem and Its Applications in Machine Learning* [12] 這篇研究的觀點，其認為在神經網路中「壓縮」會反映到資料在空間中的類聚現象。我們推測，1×1 的瓶頸層過度壓縮了輸入圖片，導致資訊缺失，故選擇採用 t-SNE（t-Distributed Stochastic Neighbor Embedding）降維技術對瓶頸層特徵做處理，此工具能將高維的特徵向量映射至二維平面，並保持樣本間的局部鄰近關係。

具體而言，我們將生成器瓶頸層輸出的 $n \times n \times C$ 特徵圖展平（Flatten），透過 t-SNE 投影後觀察不同字符的特徵分佈。若同一字符的樣本在二維平面上能夠聚集成簇（Cluster）且不同簇有明顯分隔，則代表模型成功學會了區分不同字符的特徵。反之若雜亂無章，則代表資訊在壓縮過程中流失過多（過度壓縮）。

(四) 線性探針測試

為了進一步量化模型對「幾何結構」的理解程度，我們引入了 **Linear Probe** 測試。此方法常用於自監督學習，旨在探測凍結的特徵層中是否隱含了顯性的空間資訊。

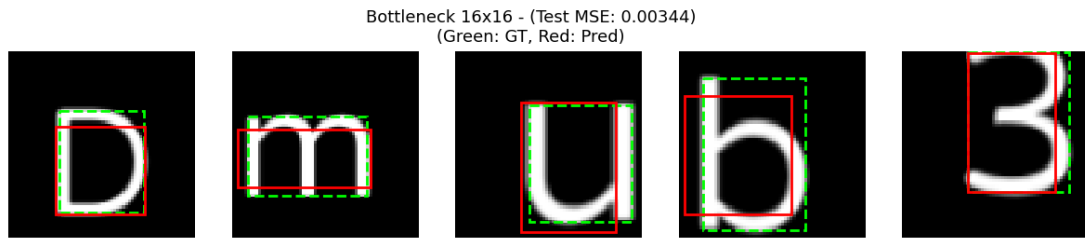


圖 21：Linear Probe 測試方法示意圖

具體而言，我們凍結（Freeze）生成器中編碼器的參數，移除解碼器改在瓶頸層後加一個簡單的線性回歸層（**Linear Layer**）並進行訓練。該層的任务是僅憑瓶頸層特徵，預測輸入字體的邊界框座標（ $[x_{min}, y_{min}, x_{max}, y_{max}]$ ）若此線性層能夠準確預測出文字的位置與大小（即測試集 MSE 誤差低），對應到此時瓶頸層保留的空間幾何資訊的準確與程度。反之若誤差較高，則說明空間資訊在壓縮過程中出現缺失。

(五) 測試結果

Bottleneck size, $n = 1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$

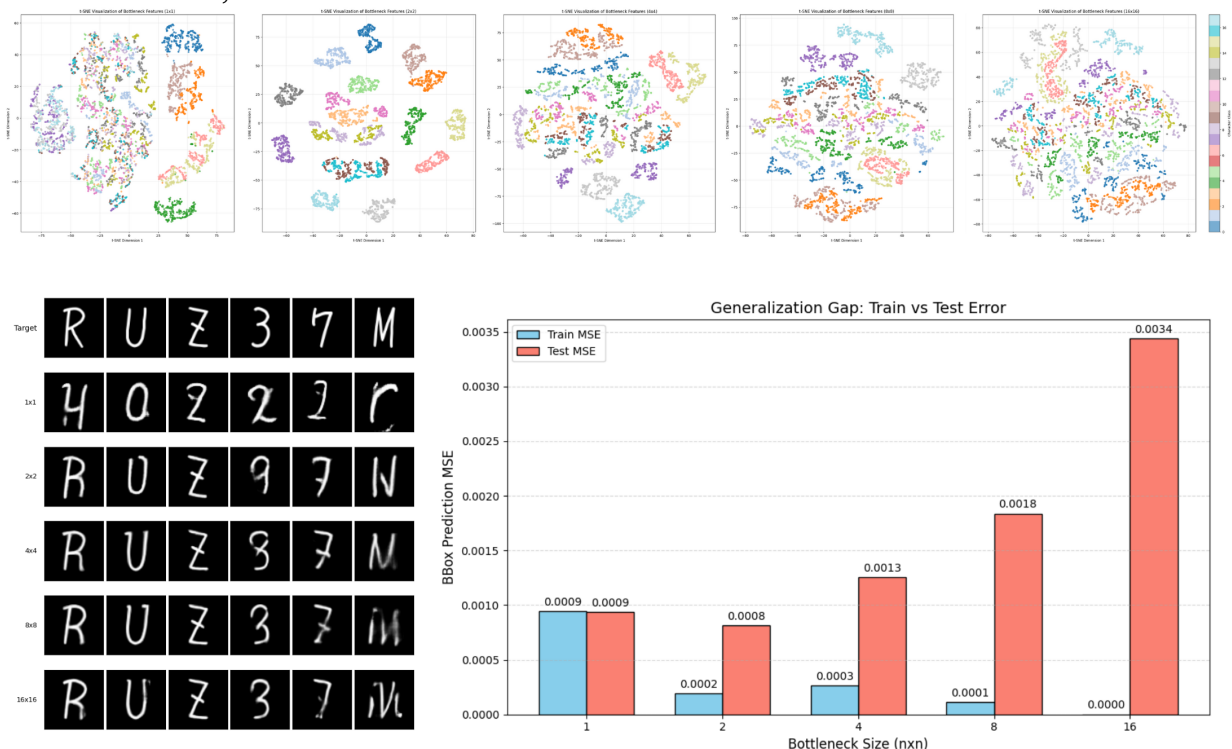


圖 22：字體 A 中不同瓶頸層的生成圖像與 t-SNE、Linear Probe 測試對比

Bottleneck size, $n = 1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$

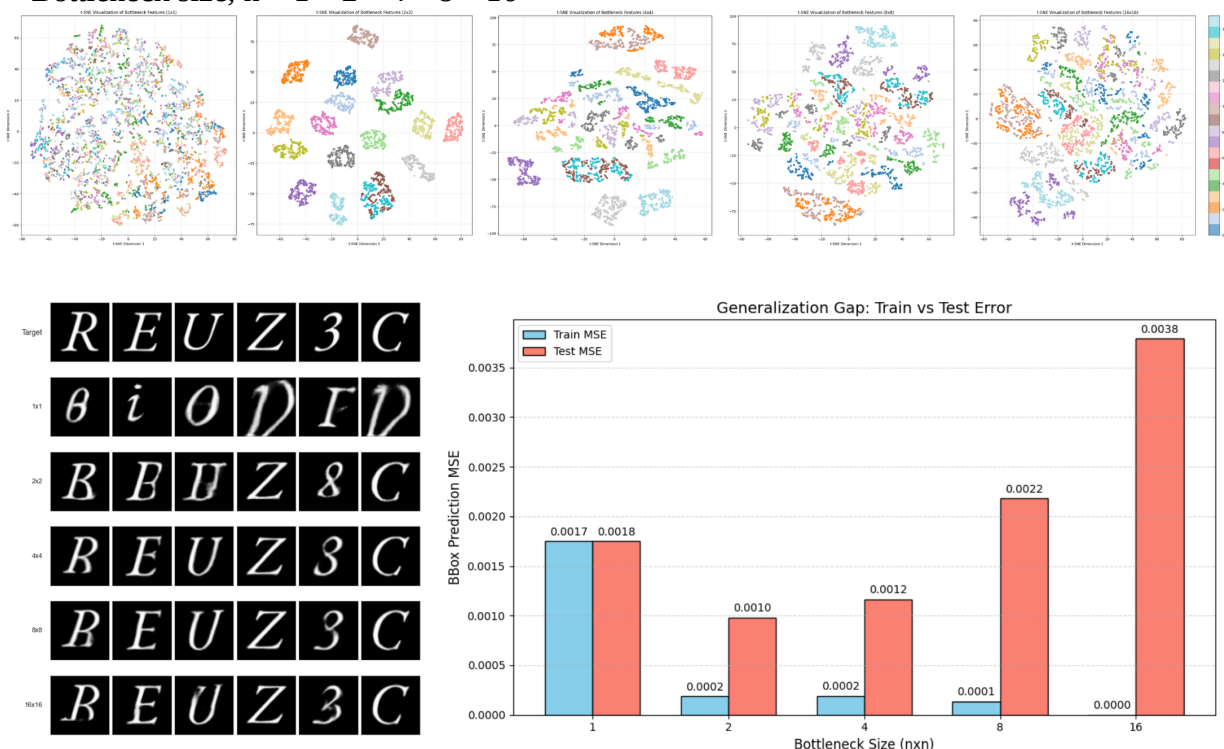


圖 23：字體 C 中不同瓶頸層的生成圖像與 t-SNE、Linear Probe 測試對比

依據特徵視覺化以及遮蔽測試的觀察可做出「瓶頸層的確在把輸入壓縮提取特徵時攜帶了一定的位置資訊」，在此基礎上進一步解釋圖 22 及圖 23 的 t-SNE 和 Linear Probe 測試結果，可得出以下結論：

1. 過小的瓶頸層（ 1×1 ）導致的結構錯誤

對於瓶頸層為 1×1 時生成結果出現字符結構錯誤及字符混淆的現象，可對應到其 t-SNE 分群中 $n=1$ 時分群失敗的情況（如圖 22、23 左上）。而在 linear probe 測試中也能發現其測試集 MSE 值是略高的，這足以說明 1×1 的瓶頸層造成的「資訊過度壓縮」是導致其生成效果非常的不理想的原因。

2. 過大的瓶頸層尺寸（ $n > 8$ ）的負面影響

當瓶頸層過大（ $n > 8$ ）其分群效果較差，分群交界較不明顯（如圖 22、23），相同字符出現了分群不同的現象（同顏色資料點出現多於一個族群），其原因是過大的瓶頸層沒辦法確實的提取出不同字符的關鍵資訊導致過多的雜訊出現。Linear Probe 測試中則出現過擬合現象，在訓練集 MSE 極小說明確實保留了更具體的結構，但測試集 MSE 極大證明了其無法將資訊精煉成利於後續操作與泛化的特徵。總結來說當 $n > 8$ 時，模型依賴輸入筆畫的完整結構保留來輸出結構正確的圖像，但在風格變換上效果不如 $n \leq 8$ 的瓶頸層。

3. 兩項測試與實際生成效果出現的部分矛盾

不過，除了 $n=1$ 的過度壓縮以外，能發現 2×2 的瓶頸層能在這兩項測試取得較好成績，這與生成效果在 $n=4$ 及 $n=8$ 才有最優表現的現象不符。此矛盾發生的原因如下：

Linear Probe 測試中，邊界框僅代表文字的「粗略位置」，回歸層能容易的從 2×2 的特徵層提取出這個低維度的特徵（可從瓶頸層遮蔽測試得知）。同樣的，t-SNE 測試中最終只需要歸類為 2 個維度（軸向）來區分字符，這對 2×2 的瓶頸層來說應也是容易且較少干擾的。

不過當任務回到「生成精細字體」時， 4×4 或 8×8 的特徵中本來成為干擾的高頻筆畫細節資訊，得以在更深層的解碼器還原下，協助生成出筆觸自然且細緻的字體。也就是說，過小的瓶頸層雖能精煉出一定的位置資訊，但捨去了部分對生成字體有幫助的高頻細節，這與 RAC-Font [10] 中提出的解釋不謀而合。

（六）模型層數差異

由於控制變因的選擇造成 4×4 的參數量大於 1×1 的參數量，為了排除參數量造成的誤差影響，本研究針對 $n=4$ 的情況進行了模型層數的調整，進一步確認影響結果的原因。

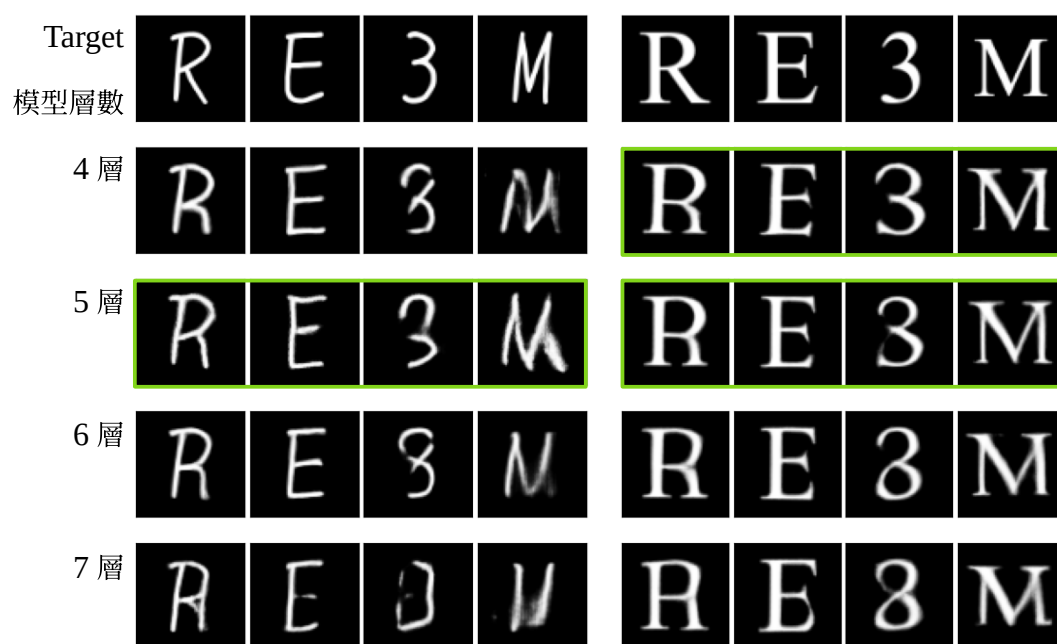


圖 24：不同模型層數應用於 4×4 之瓶頸層上對生成圖片的影響

由圖 24，可以發現當層數降至 4 層時，此時參數量已遠小於 $n=1$ 的 6 層模型，生成效果並沒有可見的降低（甚至更佳）。這說明瓶頸層尺寸的選擇相較模型的深度（參數量）確實是影響生成結果的關鍵。

四、最佳化損失函數權重配置

如圖 25，在瓶頸層尺寸為 1×1 時，移除 L_1 損失會造成模型輸出符合目標字體風格但與來源字符無關的圖像（模式崩潰）。此時應用較高的 L_1 損失權重可改善此情況，但其結構以及筆畫細節仍不穩定。

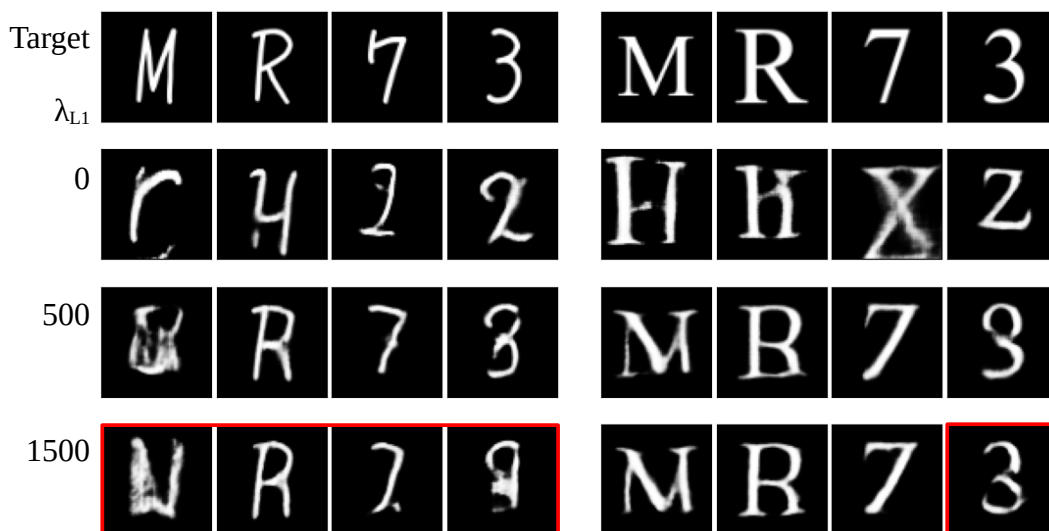


圖 25：不同的 L_1 損失權重 λ_{L_1} 對瓶頸層尺寸為 1×1 的模型生成效果之影響

不同的是，當使用較大（ $n \geq 4$ ）的瓶頸層，不添加 L_1 損失也能生成正確的字體結構。此時過高的 λ_{L_1} 反而會造成經典的模糊現象（如表 3 及圖 26）（表 3 中部分高 λ_{L_1} 之組合評分仍好是因為量化指標傾向獎勵空間平均對齊的模糊圖像，而非位置稍有偏差的銳利圖像）。

註：WGAN-GP 架構中對抗損失 L_{adv} 為較高的純量分數，故 λ_{L_1} 設定需較高。

表 3：瓶頸層為 4×4 時不同 λ_{L_1} 下字體 A 與字體 B 於測試集之 SSIM 與 LPIPS 評分

λ_{L_1}	SSIM-Font A $\uparrow$	LPIPS-Font A $\downarrow$	SSIM-Font B $\uparrow$	LPIPS-Font B $\downarrow$
0	0.755979	0.114556	0.809204	0.0730651
1000	<u>0.764766</u>	0.106777	<u>0.800040</u>	<u>0.0746161</u>
1500	0.763568	<u>0.108034</u>	0.795854	0.0775417
2500	0.767562	0.108826	0.794992	0.0759182



圖 26：不同的 L1 損失權重 λ_{L1} 對瓶頸層尺寸為 4×4 的模型生成效果之影響

相較前人的研究 [5] [6] [8] [10] 中須採用 L1 損失主導或是相等權重的配置來約束生成結構（如 RAC-Font [10] 將其 L1 損失的權重設為對抗損失的 100 倍），本研究得以直接移除容易帶來模糊的 L1 損失並得到較好的結果，這個發現是令人驚喜的。我們意識到這個結果應是由 WGAN-GP 架構所帶的提升。相比於傳統 GAN 使用了在結構錯誤（分布重疊度低）時無法給出有效梯度的 JS 散度，WGAN-GP 使用的 Wasserstein 距離能在結構錯誤較大時持續提供梯度，自然地引導模型修正幾何結構，降低對 L1 損失的依賴，這在其他研究中是尚未提到的有價值發現。

五、 評估跨語言字體生成之可行性

由於中文字體訓練樣本天然的就比英文多（對於本研究，英文僅 62 個英數，中文則挑選了常見的 180 個部首或簡單字符）且漢字中相同結構重複使用的情況非常普遍，使得「同步變換增強」能在相同的結構出現時發揮極好的效果。[7]

Source	木	天	日	建	岩	朝	皇	今	聞	真	國	愛	極
Target	木	天	日	建	岩	朝	皇	今	聞	真	國	愛	極
Generated	木	天	日	建	岩	朝	皇	今	聞	真	國	愛	極
	訓練集資料												
Source	木	天	日	建	岩	朝	皇	今	聞	真	國	愛	極
Target	木	天	日	建	岩	朝	皇	今	聞	真	國	愛	極
Generated	木	天	日	建	岩	朝	皇	今	聞	真	國	愛	極

圖 27：將英文字體實驗中總結的最佳配置直接應用於中文字體生成之效果

圖 27 左側的「木、天、日」三字為訓練集代表字，與英數字集相同，模型皆能在各面向完全模仿目標字體。訓練集中含類似部件的「岩、朝、皇、今、國」幾字，整體結構正確，視覺風格也符合目標字體，僅出現少數筆畫風格錯誤或模糊的情況。「愛、極」兩字，則因結構較特別（訓練集中從未出現）且筆畫密集，故出現嚴重模糊情況，推測這也與一開始選擇的 64×64 的圖片尺寸過小有關，是未來必須改善的問題。此外 RAC-Font [10] 提到的針對細部筆畫結構做額外訓練的方法或許也是值得借鑑的。

陸、 結論

一、 WGAN-GP 架構的高穩定性

本研究驗證了 WGAN-GP 在字體生成任務中的高度穩定性。相較於傳統 GAN 易發生的梯度消失、模式崩潰或生成模糊等問題，WGAN-GP 透過 Wasserstein 距離與梯度懲罰，確保了判別器在訓練全程皆能提供有效的梯度指引。這不僅使的模型得以在無 L1 損失輔助下自我修正結構，更為後續的瓶頸層與資料增強實驗提供了穩定的基礎架構。

二、 同步變換增強的邊界效應

「同步變換增強（Sync. Aug.）」再次被證實能有效提升模型對陌生字符的泛化能力。然而增強參數存在明顯的「甜蜜點」，過度的變換會導致結構裁切，形成誤導性的無效數據。此外，隨機遮罩與旋轉變換在字體生成任務中被證實效果不顯著甚至是具負面的。

三、 瓶頸層空間維度是結構完整性的關鍵

相對於認為應將特徵壓縮至 1×1 以提取純粹特徵的觀點，本研究透過 t-SNE 與 Linear Probe 探針實驗說明 1×1 的過度壓縮會導致空間位置資訊的流失，進而引發結構崩塌或字符混淆。也透過同樣方法解釋了過大的瓶頸層保留過多雜訊的缺陷，最終得出在不使用 U-Net 跳躍連接或其他輔助的情況下， 4×4 及 8×8 的瓶頸層能有最佳的生成效果。

四、 損失函數與架構的連動關係

瓶頸層與模型的架構設計直接影響了損失函數的配置策略。當瓶頸層保留了足夠的空間資訊（如 $n \geq 4$ ），並透過 WGAN-GP 架構的 Wasserstein 距離引導模型時，可不再依賴 L1 像素損失或 U-Net 架構來鎖定結構。這允許我們得以移除易造成模糊現象的 L1 損失權重，從而使生成圖像在結構準確的同時保有銳利的筆畫細節。此發現也證實在適當的架構設計下，單純的對抗式學習足以捕捉複雜的幾何結構，相較於過往研究依賴 L1 損失與 U-Net 跳躍連接的觀點，是有價值的發現。

五、 跨語言生成的潛力與限制

本研究的方法在英文與數字資料集驗證成功後，遷移至中文字體生成亦展現了高度可行性。利用漢字部件重複比例較高的特性，僅使用 180 個字符訓練，模型即能推論出從未見過的漢字，實現少樣本生成。不過受限於目前的圖像解析度（ 64×64 ），在處理筆畫密集或結構複雜的漢字（如愛、極等）時，仍存在筆畫沾黏或模糊等限制。未來除了提高圖片解析度外，也將嘗試使用類似注意力機制的方法，針對筆畫細節做對應的訓練。

柒、 參考文獻

- [1] Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative adversarial nets. *Advances in neural information processing systems*, 27.
- [2] Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*.
- [3] Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein generative adversarial networks. In *International conference on machine learning* (pp. 214-223). PMLR.
- [4] Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. C. (2017). Improved training of wasserstein gans. *Advances in neural information processing systems*, 30.

- [5] Isola, P., Zhu, J. Y., Zhou, T., & Efros, A. A. (2017). Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1125-1134).
- [6] Tian, Y. (2017). *Zi2zi: Master Chinese Calligraphy with Conditional Adversarial Networks*. Github. <https://github.com/kaonashi-tyc/zi2zi>
- [7] Wen, C., Pan, Y., Chang, J., Zhang, Y., Chen, S., Wang, Y., ... & Tian, Q. (2021). Handwritten Chinese font generation with collaborative stroke refinement. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision* (pp. 3882-3891).
- [8] Chen, X., Xie, Y., Sun, L., & Lu, Y. (2022). Dgfont++: Robust deformable generative networks for unsupervised font generation. *arXiv preprint arXiv:2212.14742*.
- [9] Pathak, D., Krahenbuhl, P., Donahue, J., Darrell, T., & Efros, A. A. (2016). Context encoders: Feature learning by inpainting. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2536-2544).
- [10] Lee, J. S., & Choi, H. C. (2024). One-shot font generation via local style self-supervision using Region-Aware Contrastive Loss. *Journal of King Saud University-Computer and Information Sciences*, 36(4), 102028.
- [11] Zeng, J., Chen, Q., Liu, Y., Wang, M., & Yao, Y. (2021, May). Strokegan: Reducing mode collapse in chinese font generation via stroke encoding. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 35, No. 4, pp. 3270-3277).
- [12] Goldfeld, Z., & Polyanskiy, Y. (2020). The information bottleneck problem and its applications in machine learning. *IEEE Journal on Selected Areas in Information Theory*, 1(1), 19-38.